



Develop microservices with Spring Boot

A.A. 2025/2026

Service-oriented Software Engineering

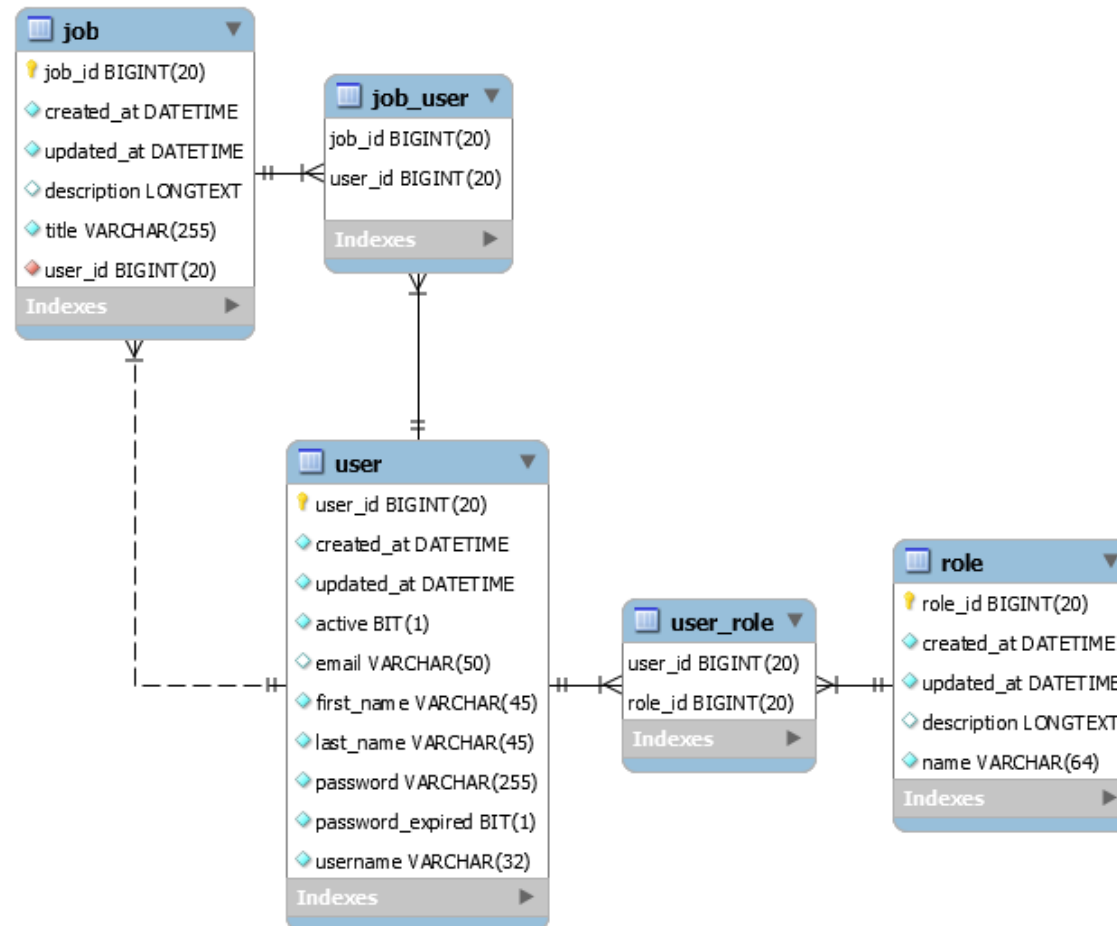
Gianluca Filippone

gianluca.filippone@gssi.it

Monolithic application

OpenJob

- › Platform that allows people to publish or apply for job offers



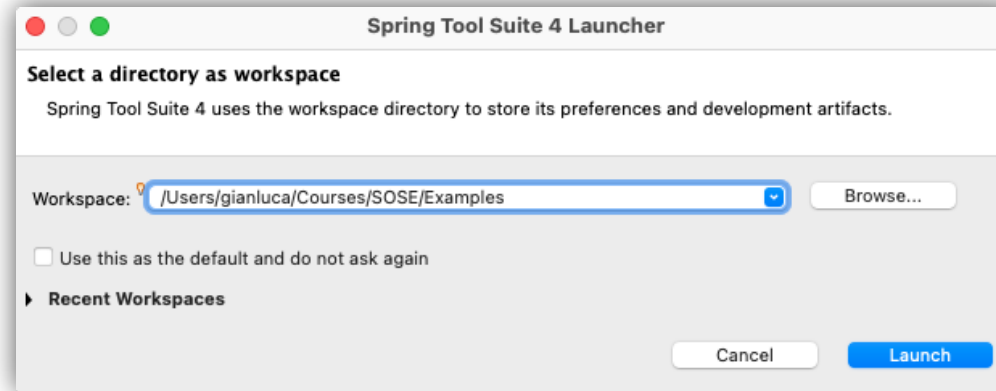
Build the application through Spring Boot

Ingredients

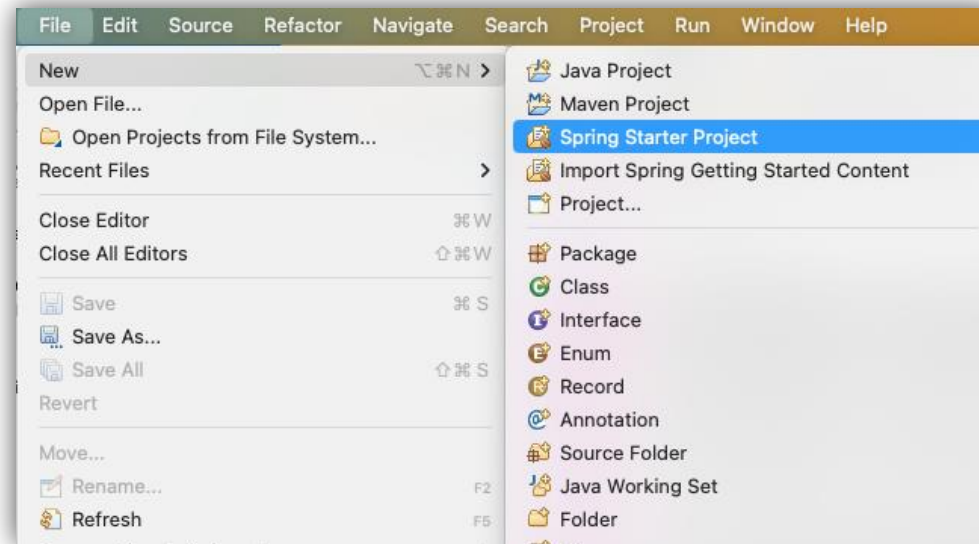
- › Java Development Kit (17 or later)
- › Apache Maven
- › Spring Tools 4
<https://spring.io/tools>

1: Initialize the project

1. Open Spring Tools and create a new workspace

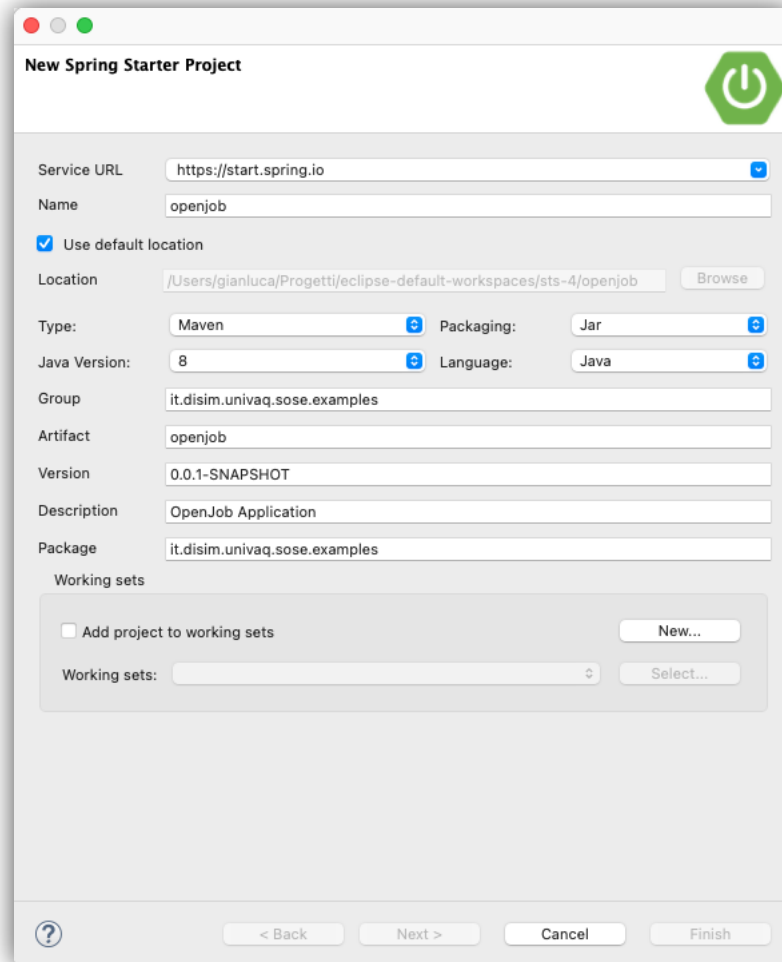


2. File -> new -> Spring Starter Project



1: Initialize the project

3. Follow the wizard



New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

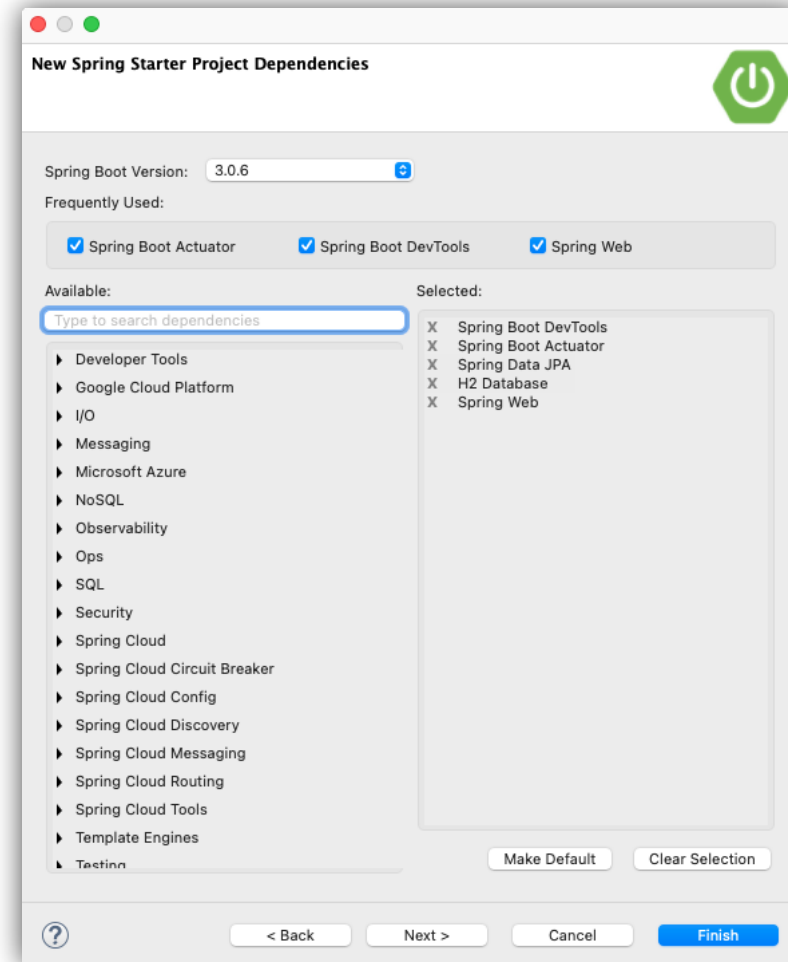
Description:

Package:

Working sets

☐ Add project to working sets

Working sets:



New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☒ Spring Boot Actuator ☒ Spring Boot DevTools ☒ Spring Web

Available:

- Developer Tools
- Google Cloud Platform
- I/O
- Messaging
- Microsoft Azure
- NoSQL
- Observability
- Ops
- SQL
- Security
- Spring Cloud
- Spring Cloud Circuit Breaker
- Spring Cloud Config
- Spring Cloud Discovery
- Spring Cloud Messaging
- Spring Cloud Routing
- Spring Cloud Tools
- Template Engines
- Testing

Selected:

- X Spring Boot DevTools
- X Spring Boot Actuator
- X Spring Data JPA
- X H2 Database
- X Spring Web

1: Initialize the project

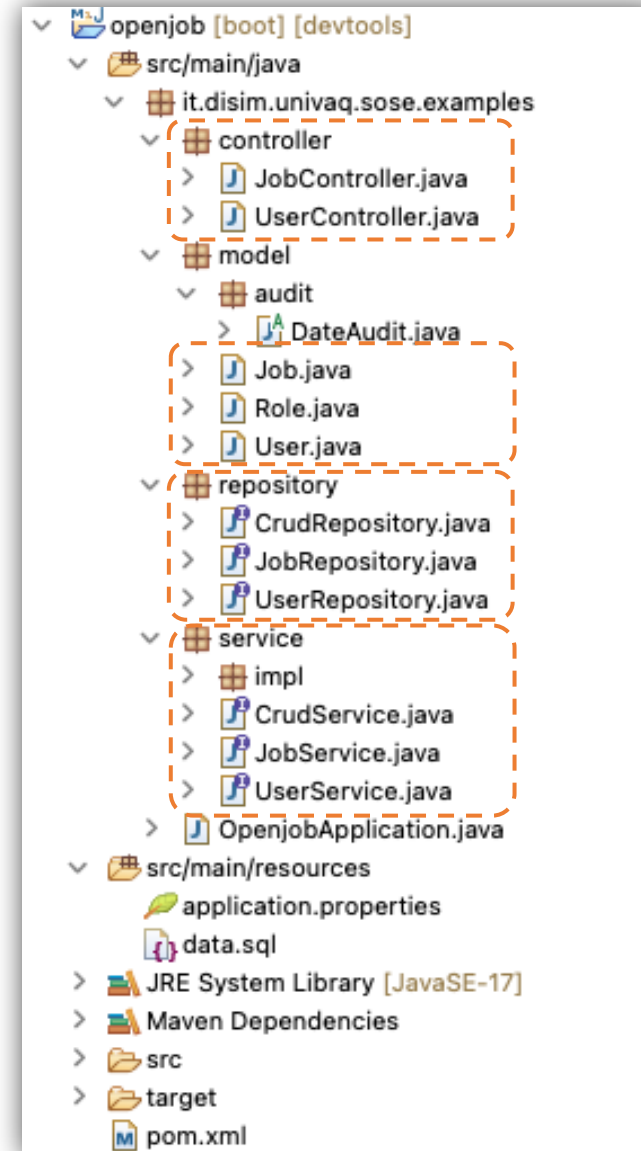
Selected dependencies

- › Spring Boot DevTools
 - › Provides fast application restarts, LiveReload, and configurations for enhanced development experience
- › Spring Boot Actuator
 - › Supports built in (or custom) endpoints to monitor and manage the application (e.g., health, metrics, sessions, etc.)
- › Spring Data JPA
 - › Persist data with Java Persistence API using Spring Data and Hibernate
- › H2 Database
 - › Fast, zero-configuration in-memory database
- › Spring Web
 - › Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

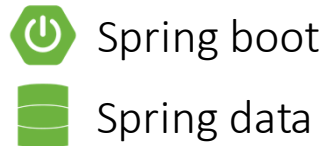
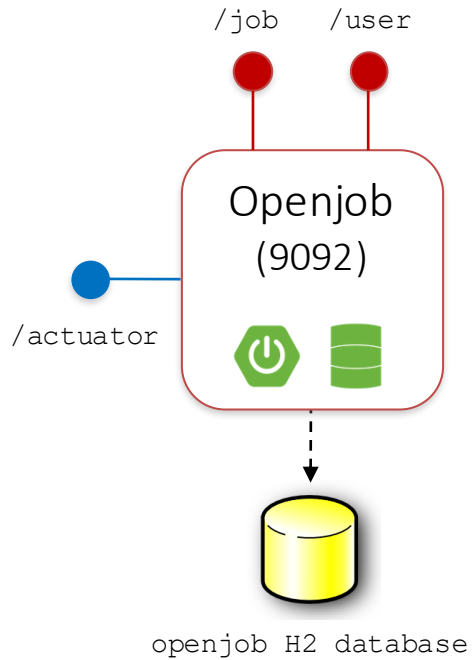
2: Implement the application

- › The result we want to obtain
- › Implement controllers, services etc.

<https://github.com/gianlucafilippone/springboot-microservices-development/mobolithic-application/openjob>



3: Result



Available Endpoints

> /actuator

- › [GET] <http://localhost:9092/actuator/health>
- › [GET] <http://localhost:9092/actuator/info>

> /user

- › [GET, POST, PUT] <http://localhost:9092/user>
- › [GET, DELETE] <http://localhost:9092/user/id/<ID>>
- › [GET] <http://localhost:9092/user/username/<USERNAME>>

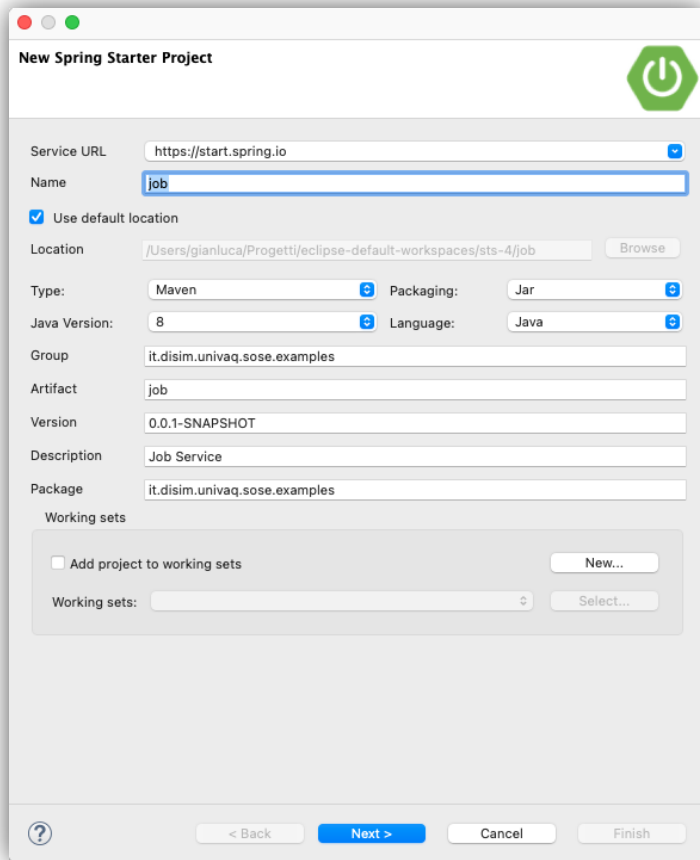
> /job

- › [GET, POST, PUT] <http://localhost:9092/job>
- › [GET, DELETE] <http://localhost:9092/job/<ID>>
- › [GET] <http://localhost:9092/job/apply/<USERNAME>/<ID>>

Migrate the application to microservices

Realize User and Job microservices

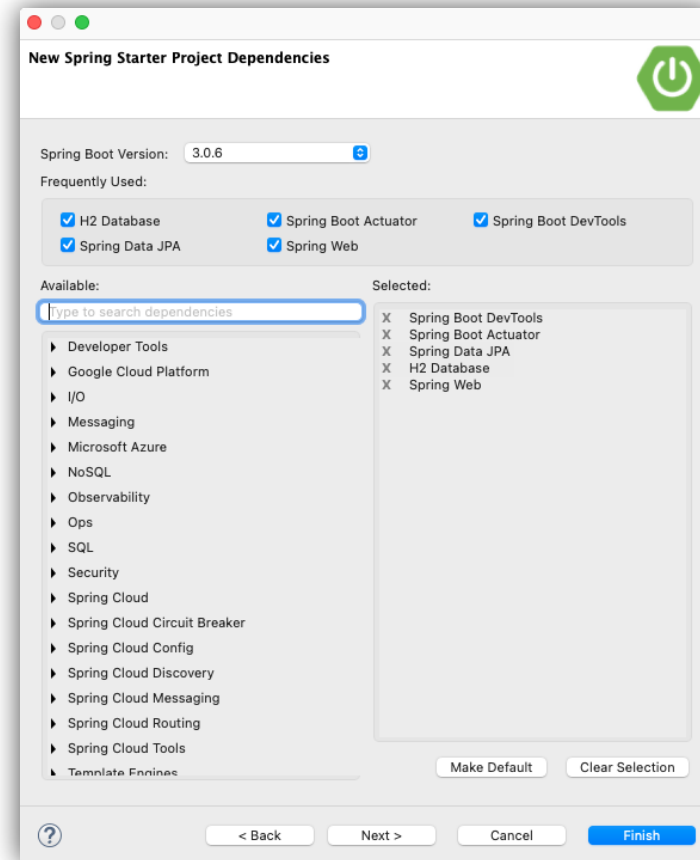
- › Initialize two applications using the wizard: user and job
 - › Same dependencies as the monolith (for now)



The 'New Spring Starter Project' dialog is shown. It contains the following fields and options:

- Service URL: `https://start.spring.io`
- Name: `job`
- ☒ Use default location
- Location: `/Users/gianluca/Progetti/eclipse-default-workspaces/sts-4/job` (with a 'Browse' button)
- Type: `Maven` (dropdown), Packaging: `Jar` (dropdown)
- Java Version: `8` (dropdown), Language: `Java` (dropdown)
- Group: `it.disim.univaq.sose.examples`
- Artifact: `job`
- Version: `0.0.1-SNAPSHOT`
- Description: `Job Service`
- Package: `it.disim.univaq.sose.examples`
- Working sets section:
 - ☐ Add project to working sets
 - New... button
 - Working sets: (empty dropdown)
 - Select... button

Navigation buttons at the bottom: ? icon, < Back, Next > (highlighted in blue), Cancel, Finish.



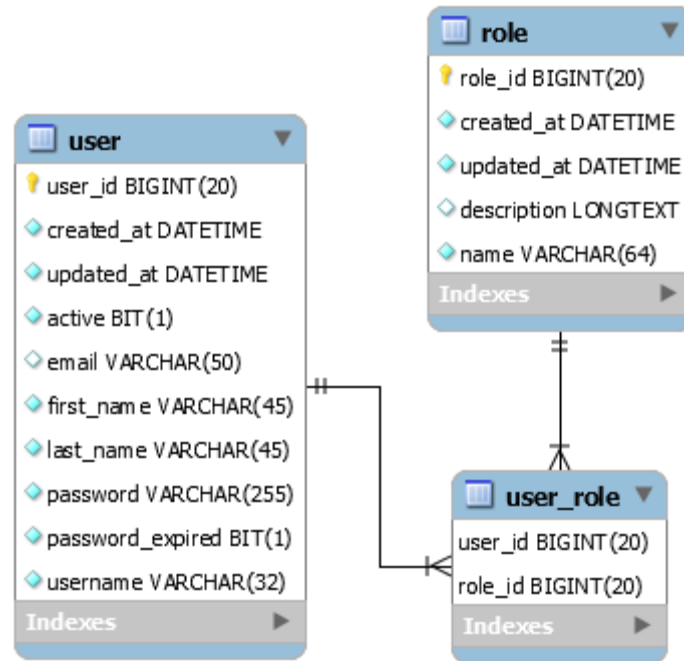
The 'New Spring Starter Project Dependencies' dialog is shown. It contains the following fields and options:

- Spring Boot Version: `3.0.6` (dropdown)
- Frequently Used section:
 - ☒ H2 Database
 - ☒ Spring Data JPA
 - ☒ Spring Boot Actuator
 - ☒ Spring Web
 - ☒ Spring Boot DevTools
- Available: (tree view)
 - Developer Tools
 - Google Cloud Platform
 - I/O
 - Messaging
 - Microsoft Azure
 - NoSQL
 - Observability
 - Ops
 - SQL
 - Security
 - Spring Cloud
 - Spring Cloud Circuit Breaker
 - Spring Cloud Config
 - Spring Cloud Discovery
 - Spring Cloud Messaging
 - Spring Cloud Routing
 - Spring Cloud Tools
 - Template Engines
- Selected: (list)
 - X Spring Boot DevTools
 - X Spring Boot Actuator
 - X Spring Data JPA
 - X H2 Database
 - X Spring Web
- Make Default button
- Clear Selection button

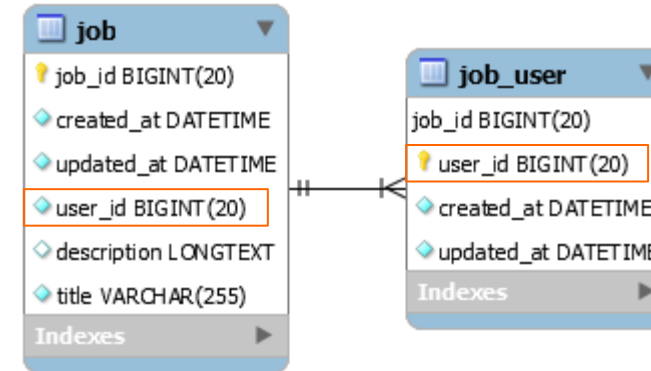
Navigation buttons at the bottom: ? icon, < Back, Next > (highlighted in blue), Cancel, Finish (highlighted in blue).

Decomposed database

› Database Schema(s)



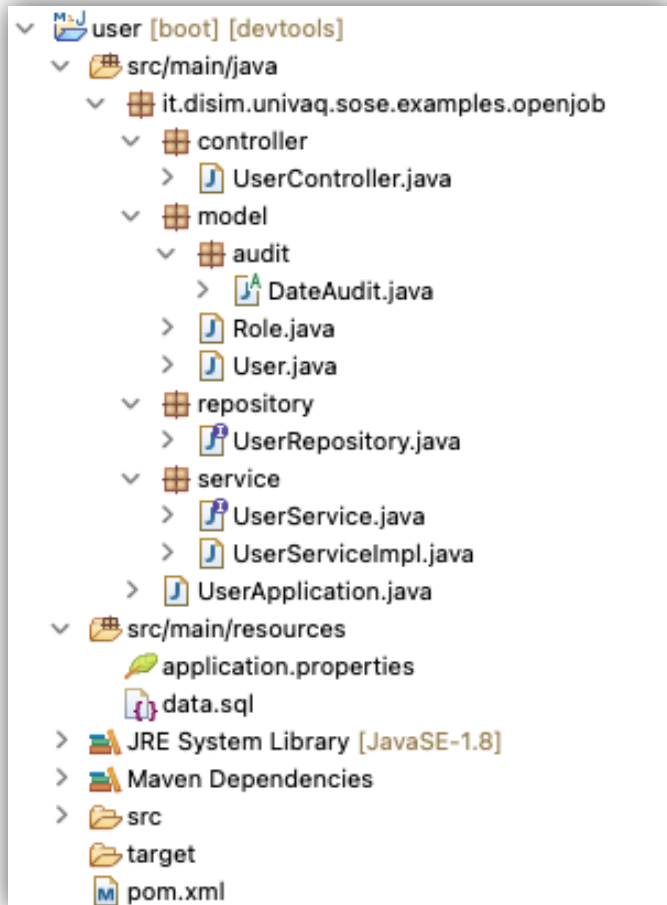
User Service schema



Job Service schema

User service

› The result we want to obtain:



› Implement controllers, services etc.

Available Endpoints

› /actuator

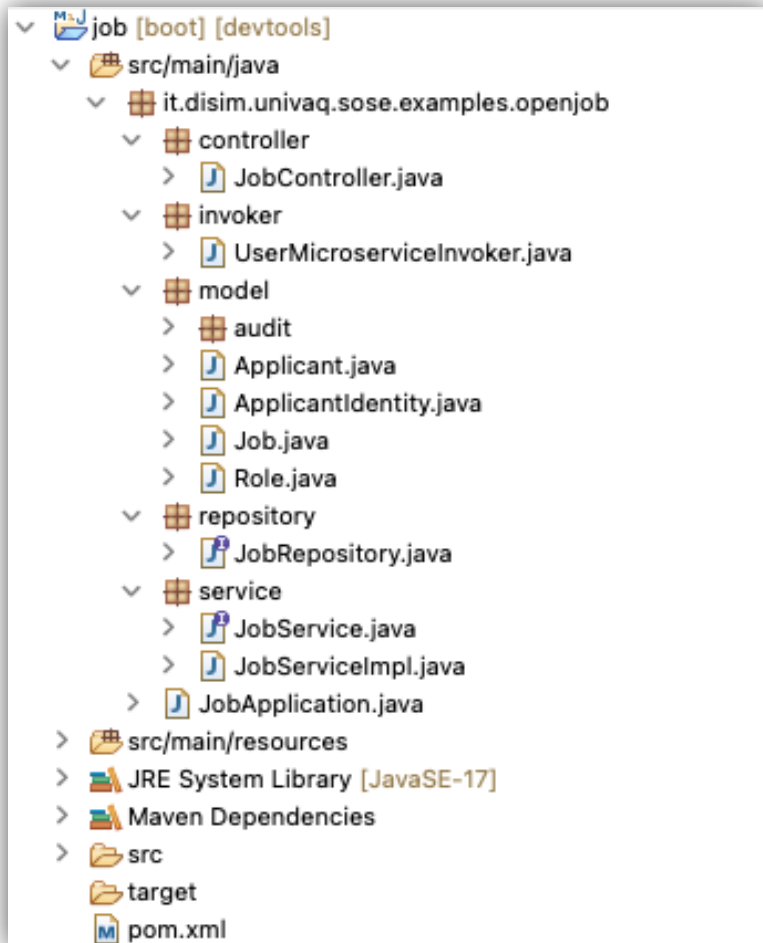
- › [GET] <http://localhost:9044/actuator/health>
- › [GET] <http://localhost:9044/actuator/info>

› /user

- › [GET, POST, PUT] <http://localhost:9044/user>
- › [GET, DELETE] <http://localhost:9044/user/id/<ID>>
- › [GET] <http://localhost:9044/user/username/<USERNAME>>

Job Service

› The result we want to obtain:



› Implement controllers, services etc.

Available Endpoints

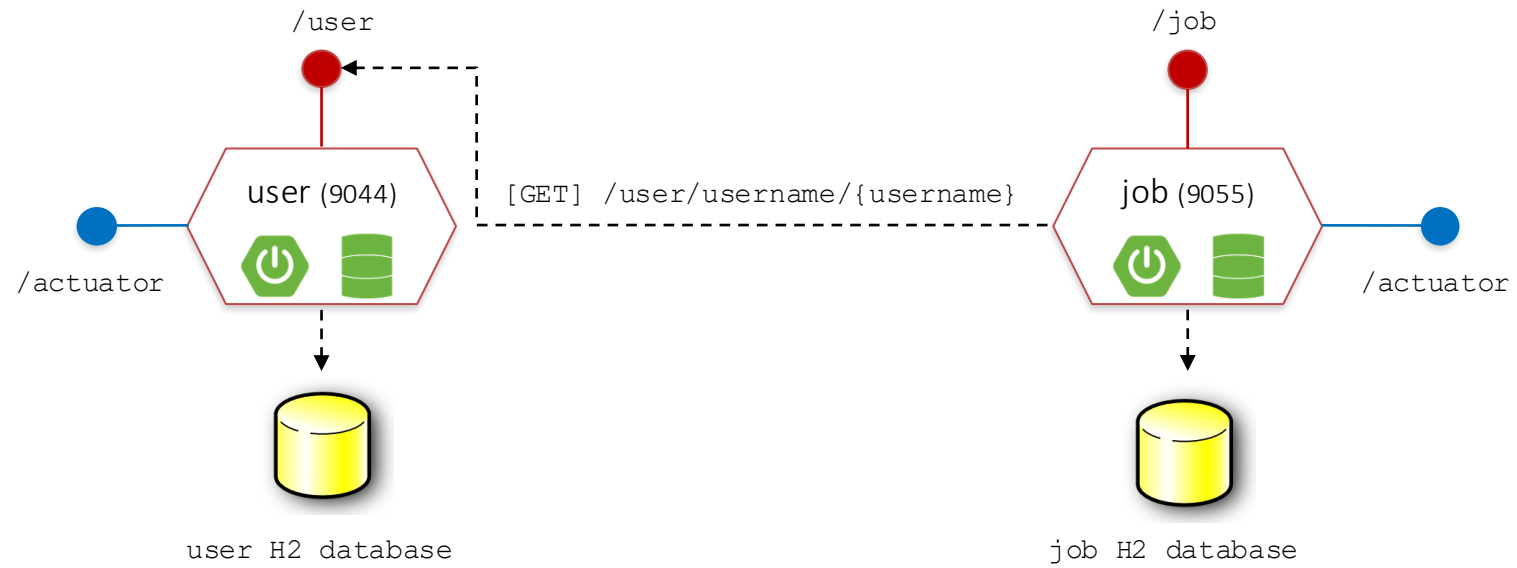
› /actuator

- › [GET] <http://localhost:9055/actuator/health>
- › [GET] <http://localhost:9055/actuator/info>

› /job

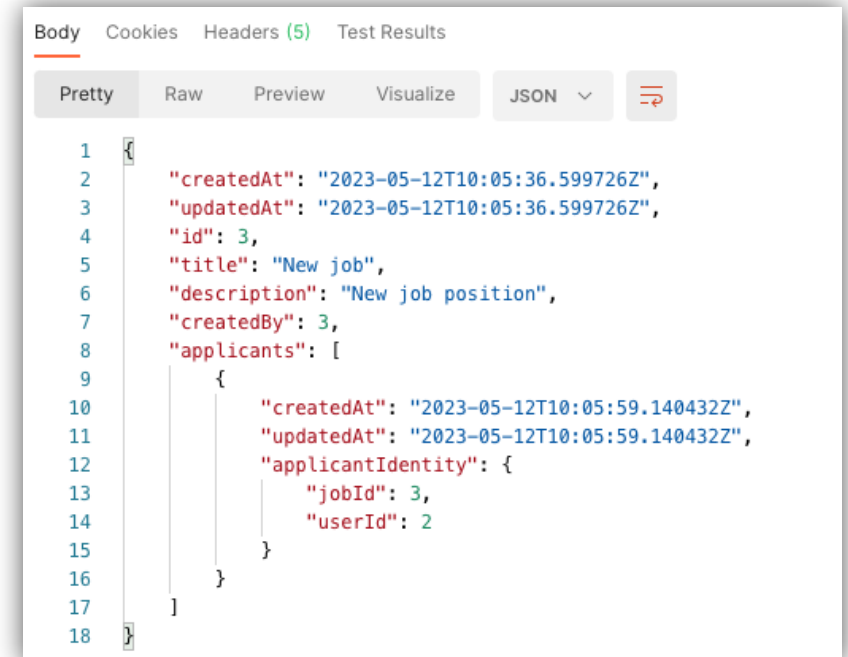
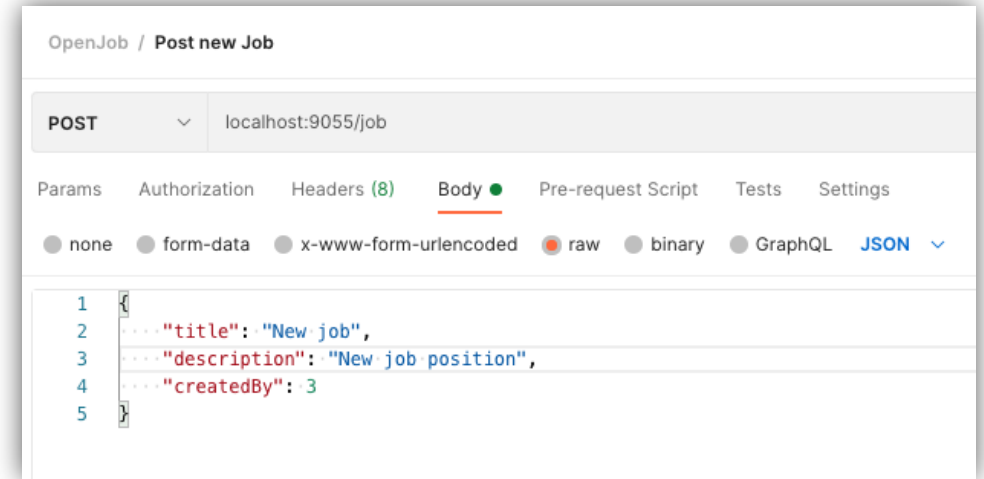
- › [GET, POST, PUT] <http://localhost:9055/job>
- › [GET, DELETE] <http://localhost:9055/job/<ID>>
- › [GET] <http://localhost:9055/job/apply/<USERNAME>/<ID>>

Result



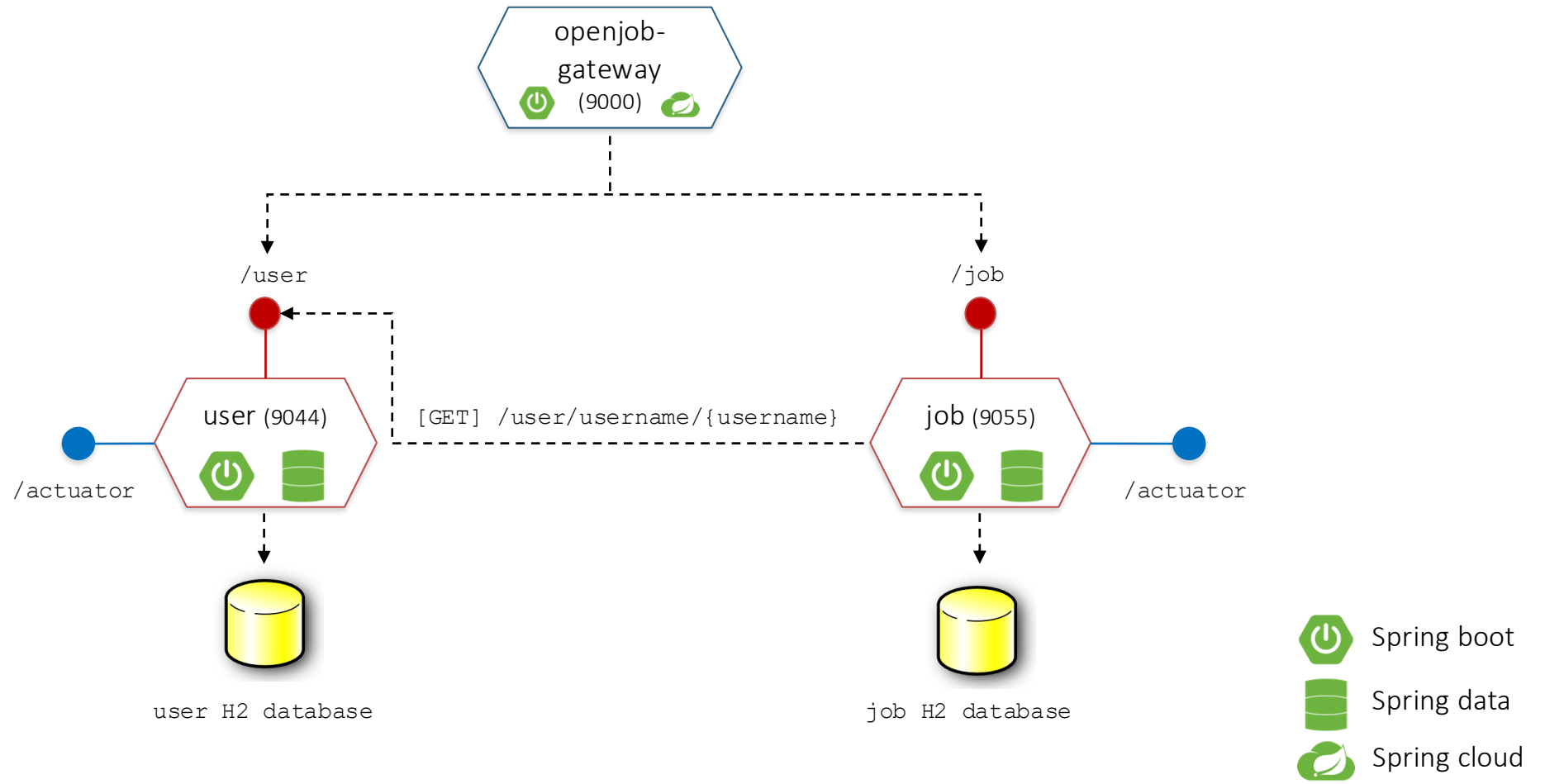
Test the application

1. Run both (micro)services
2. Get list of users and jobs
 - › [GET] `http://localhost:9044/user`
 - › [GET] `http://localhost:9055/job`
3. Post a new job offer
 - › [POST] `http://localhost:9055/job`
4. Apply for a job
 - › [GET] `http://localhost:9055/job/apply/<USERNAME>/<JOB_ID>`
5. Get the details of the applied job
 - › [GET] `http://localhost:9055/job/<JOB_ID>`



Adding an API Gateway

Architecture



Spring Cloud Gateway

› Spring Cloud

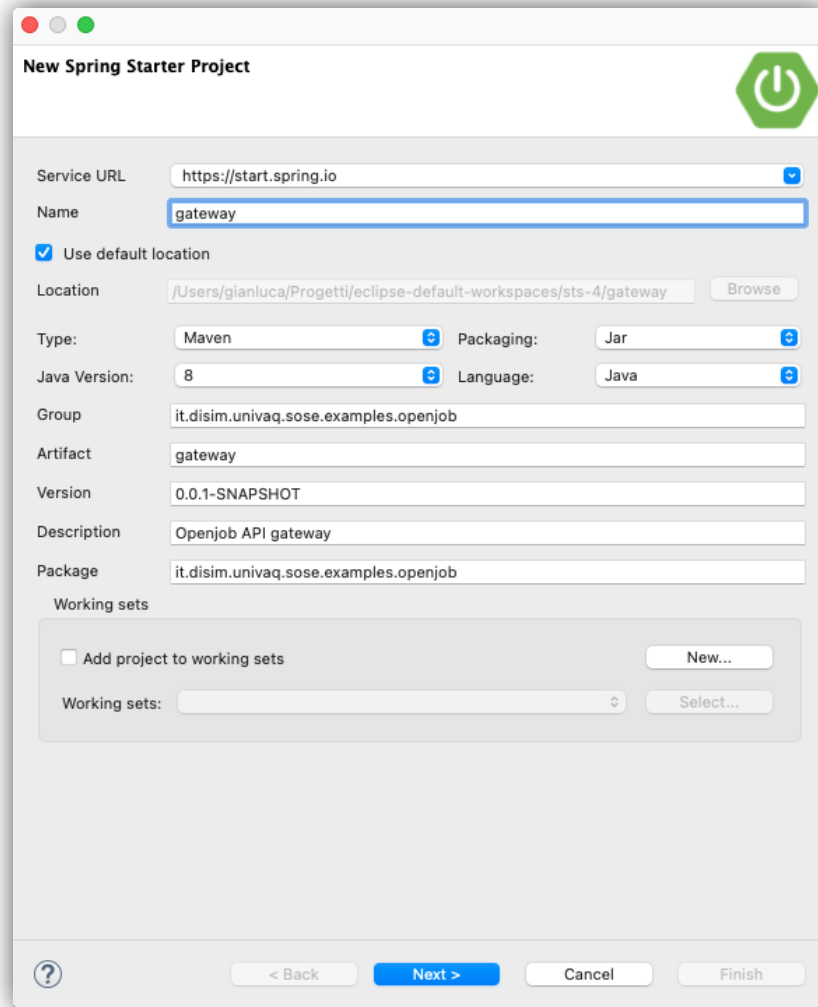
- › Spring Cloud is an umbrella containing different projects
- › Provides tools for developers to quickly build some of the common patterns in distributed systems
- › Distributed configuration, service discovery, routing, load balancing, ...

› Spring Cloud Gateway

- › Library for building an API Gateway on top of Spring WebFlux
- › Out-of-the-box routing mechanisms
- › Main configuration components:
 - Route: consists of an ID, a destination URI, a set of predicates and filters
 - Predicates: match URI, headers or parameters from the HTTP request
 - Filters: allow to modify requests and responses

Init a Spring Cloud Gateway project

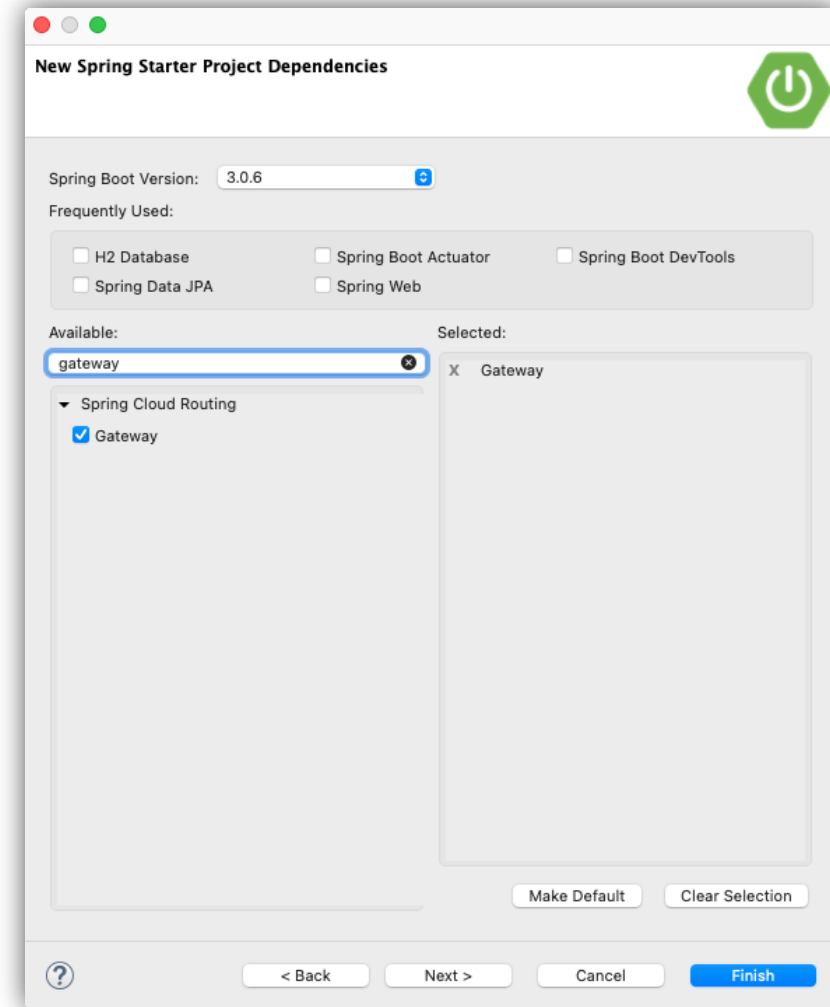
- › From the Spring Initializr or from Spring Tools
 - › Select Gateway dependency



The 'New Spring Starter Project' dialog box is shown. It contains the following fields and options:

- Service URL: `https://start.spring.io`
- Name: `gateway`
- ☒ Use default location
- Location: `/Users/gianluca/Progetti/eclipse-default-workspaces/sts-4/gateway` (with a 'Browse' button)
- Type: `Maven` (dropdown), Packaging: `Jar` (dropdown)
- Java Version: `8` (dropdown), Language: `Java` (dropdown)
- Group: `it.disim.univaq.sose.examples.openjob`
- Artifact: `gateway`
- Version: `0.0.1-SNAPSHOT`
- Description: `Openjob API gateway`
- Package: `it.disim.univaq.sose.examples.openjob`
- Working sets section: ☐ Add project to working sets, with 'New...' and 'Select...' buttons.

Navigation buttons at the bottom: `< Back`, `Next >` (highlighted in blue), `Cancel`, and `Finish`.



The 'New Spring Starter Project Dependencies' dialog box is shown. It contains the following fields and options:

- Spring Boot Version: `3.0.6` (dropdown)
- Frequently Used section: ☐ H2 Database, ☐ Spring Boot Actuator, ☐ Spring Boot DevTools, ☐ Spring Data JPA, ☐ Spring Web
- Available section: `gateway` (dropdown), ☒ Spring Cloud Routing, ☒ Gateway
- Selected section: `X Gateway`
- Buttons at the bottom right: `Make Default` and `Clear Selection`

Navigation buttons at the bottom: `< Back`, `Next >` (highlighted in blue), `Cancel`, and `Finish` (highlighted in blue).

Configuring the gateway

- › Routes we want to expose to clients:
 - › `http://localhost:9000/api/usr` for User service
 - › `http://localhost:9000/api/job` for Job service



```
1 # Port of the gateway
2 server:
3   port: 9000
4
5 # Gateway configuration
6 spring:
7   cloud:
8     gateway:
9       routes:
10         - id: user_route
11           uri: http://localhost:9044
12           predicates:
13             - Path=/api/usr/**
14           filters:
15             - RewritePath=/api/usr(?<segment>), /user${segment}
16
17         - id: job_route
18           uri: http://localhost:9055
19           predicates:
20             - Path=/api/job/**
21           filters:
22             - RewritePath=/api(?<segment>/?), ${segment}
23
```

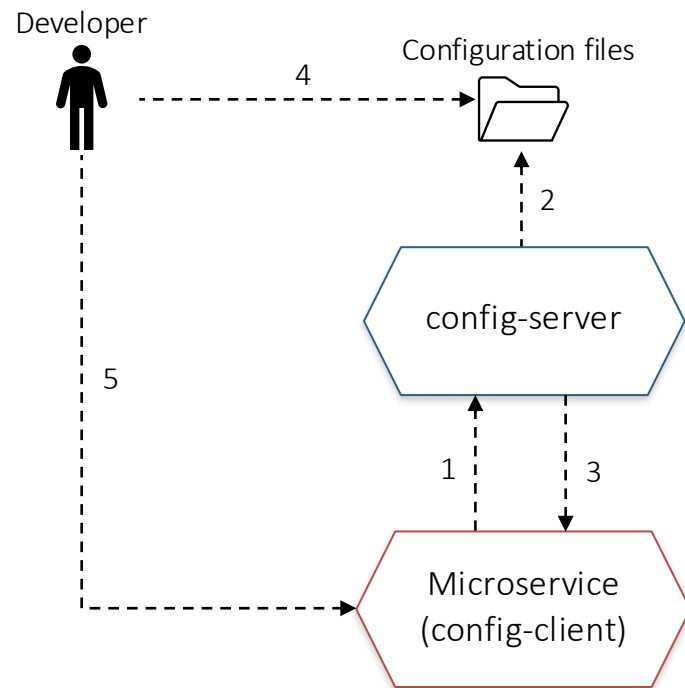
Decentralize microservice configuration

Maintaining microservices configuration

- › Maintain and manage the configuration of microservices is complex
 - › Hundreds of services with different configurations
 - › Usually done through configuration files
 - › Config updates need restart
- › Centralized mechanism for maintain configuration is convenient
 - › Store the properties of all the microservices
 - › A centralized server receives the changes and updates properties
- › Externalized configuration
 - › Spring Cloud Config provides this support

Spring Cloud Config

- › Provides support for the centralized configuration
 - › Server side (spring-cloud-config-server)
 - › Client side (spring-cloud-config-client)



At application startup

1. Microservice asks the configuration properties
2. config-service reads the configuration files
 - › From a local directory, Git, or a database
3. Configuration properties are returned to the microservice

When configuration is updated

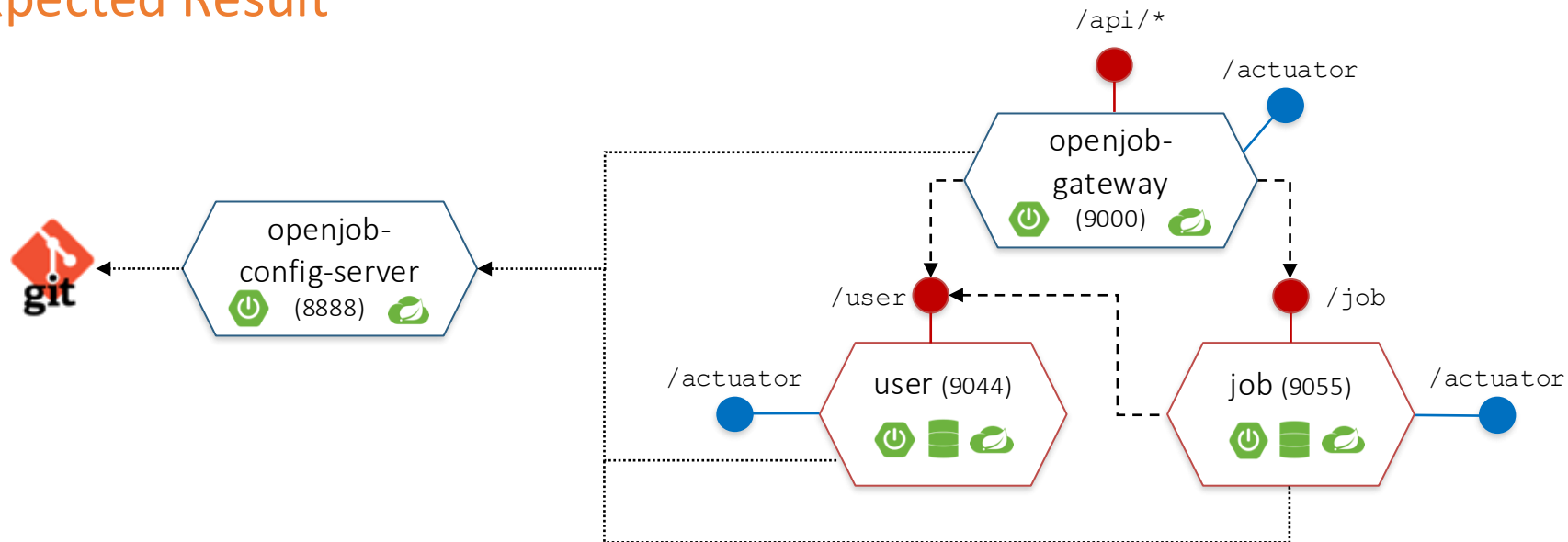
4. Developers update the configuration files
5. The update is notified to the microservice
 - › Through /actuator/refresh endpoint

Upgrade our Openjob system

Steps

1. Create a repository to store the properties files
2. Create the Spring Cloud Config server
3. Upgrade our microservices

Expected Result



Property file repository

1. Create a new git repository

- › `git init openjob-properties-repository`

2. (Optional) Import repository into Spring Tools

- › File -> Import -> Git -> Projects from Git -> Existing local repository
- › Click on "add" and select the git project folder
- › Select "import as general project"



3. Move configuration files into the repository, e.g.

- › `user/src/main/resources/application.properties >>> user-microservice-dev.properties`
- › `job/src/main/resources/application.properties >>> job-microservice-dev.properties`
- › `gateway/src/main/resources/application.yml >>> api-gateway-dev.yaml`

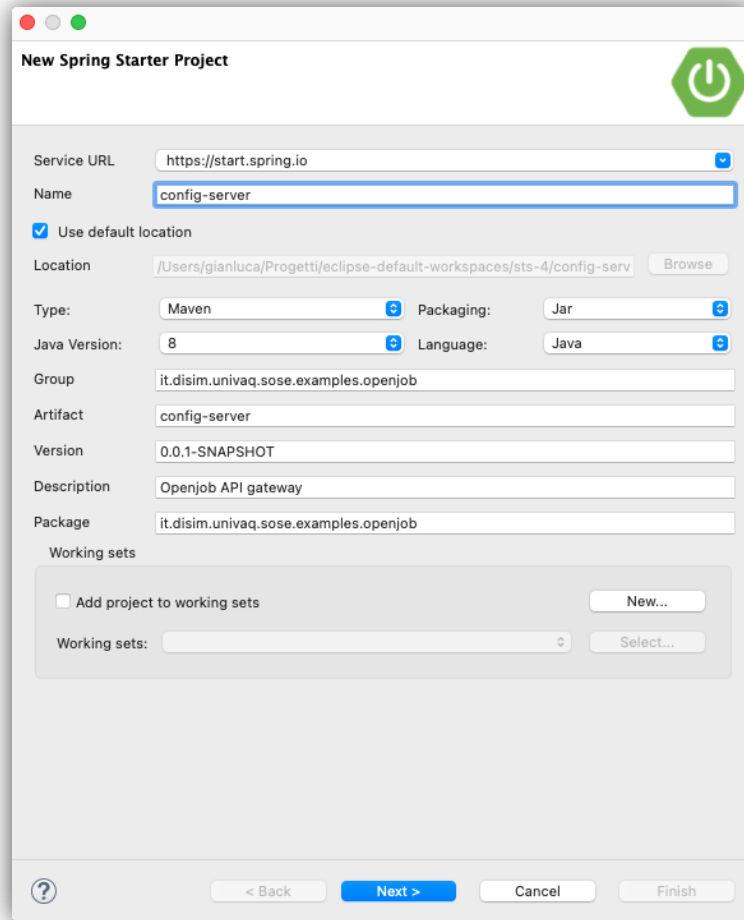
4. You can commit the files into the repository to version the configuration

- › `git add .`
- › `git commit -m 'add configuration files'`

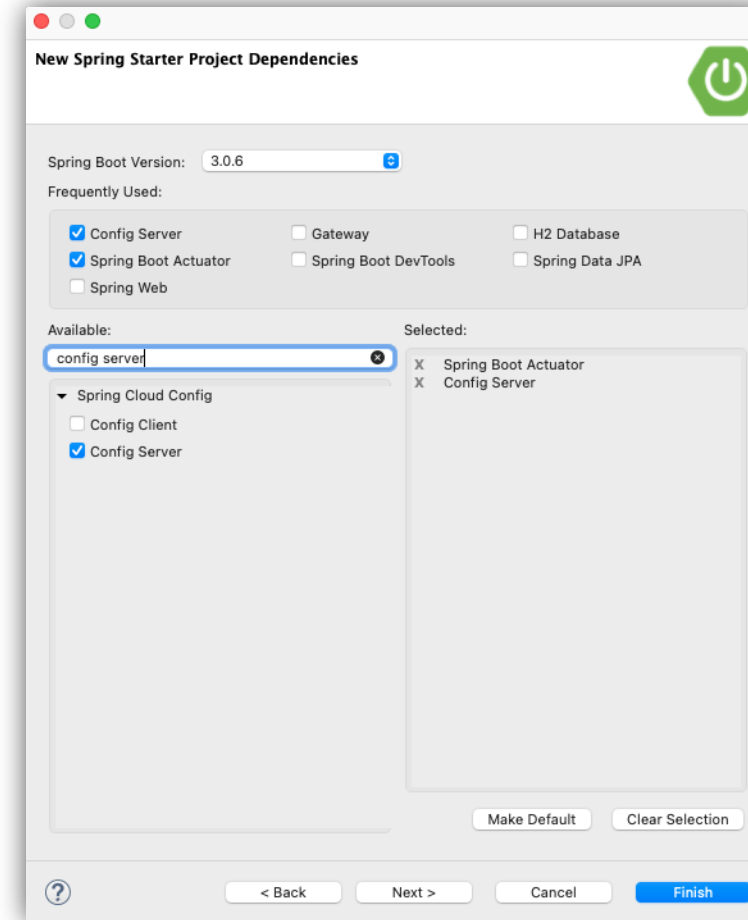
Mind these names!

Spring Cloud Config server

- › Init a new Spring Boot project
 - › Select Config Server as dependency



The 'New Spring Starter Project' dialog is shown. The 'Name' field is set to 'config-server'. The 'Service URL' is 'https://start.spring.io'. The 'Location' is '/Users/gianluca/Progetti/eclipse-default-workspaces/sts-4/config-serv'. The 'Type' is 'Maven', 'Packaging' is 'Jar', 'Java Version' is '8', and 'Language' is 'Java'. The 'Group' is 'it.disim.univaq.sose.examples.openjob', 'Artifact' is 'config-server', 'Version' is '0.0.1-SNAPSHOT', 'Description' is 'Openjob API gateway', and 'Package' is 'it.disim.univaq.sose.examples.openjob'. The 'Working sets' section has 'Add project to working sets' checked. The 'Next >' button is highlighted.



The 'New Spring Starter Project Dependencies' dialog is shown. The 'Spring Boot Version' is '3.0.6'. The 'Frequently Used' section has 'Config Server' and 'Spring Boot Actuator' checked. The 'Available' section has 'config server' selected. The 'Selected' section has 'Spring Boot Actuator' and 'Config Server' selected. The 'Finish' button is highlighted.

Spring Cloud Config server

- › Edit Application class
 - › Add @EnableConfigServer annotation
- › Add properties file
 - › Port of the server
 - › Actuator service configuration
 - › Spring cloud server configuration

```
@EnableConfigServer
@SpringBootApplication
public class ConfigServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(ConfigServerApplication.class, args);
    }

}
```

```
# Port of the server
server.port: 8888

spring.application.name=openjob-config-server

# Actuator configuration
management.endpoints.web.exposure.include=health,info
management.info.env.enabled=true

# Actuator
info.application.name=Openjob config server
info.application.description=A Server used by microservices for retrieving their configuration
info.application.version=0.0.1-SNAPSHOT

# Cloud Config ...config
spring.cloud.config.server.git.uri=file://${user.home}/Progetti/eclipse-default-workspaces/sts-4/openjob-properties-repository
#spring.cloud.config.server.git.uri=file:/// ${user.home}/config-repo On Windows, use three ///
spring.cloud.config.server.git.default-label=master
```

Spring Cloud Config server

› Test exposed endpoints

- › localhost:8888/user-microservice/dev
- › localhost:8888/job-microservice/dev
- › localhost:8888/api-gateway/dev

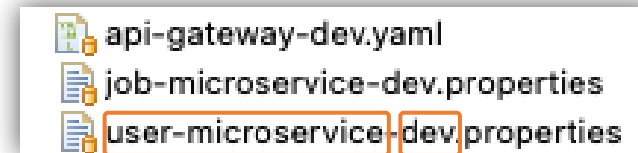
api-gateway-dev.yaml
job-microservice-dev.properties
user-microservice-dev.properties

`/ {application} / {profile} [ / {label} ]`
`/ {application} - {profile} . yml`
`/ {label} / {application} - {profile} . yml`
`/ {application} - {profile} . properties`
`/ {label} / {application} - {profile} . properties`

```
"name": "job-microservice",
"profiles": [
  "dev"
],
"label": null,
"version": "6bd9be11567117c1818bb5fef3873149a9c50398",
"state": null,
"propertySources": [
  {
    "name": "file:///Users/gianluca/Progetti/eclipse-default-workspaces/sts-4/openjob-properties-repository/job-microservice-dev.properties",
    "source": {
      "server.port": "9055",
      "spring.datasource.url": "jdbc:h2:mem:job",
      "spring.datasource.driverClassName": "org.h2.Driver",
      "spring.datasource.username": "sa",
      "spring.datasource.password": "",
      "spring.h2.console.enabled": "true",
      "spring.jpa.database-platform": "org.hibernate.dialect.H2Dialect",
      "spring.jpa.defer-datasource-initialization": "true",
      "spring.jpa.properties.hibernate.globally_quoted_identifiers": "true",
      "spring.jpa.hibernate.ddl-auto": "create",
      "management.endpoints.web.exposure.include": "health,info",
      "management.info.env.enabled": "true",
      "info.application.name": "Job",
      "info.application.description": "Job microservice",
      "info.application.version": "0.0.1-SNAPSHOT",
      "microservice.user.find.uri": "http://localhost:9044/user/username/"
    }
  }
]
```

Upgrade microservices and API Gateway

- › Update pom.xml file (see slide notes)
 - › Add dependency management for spring cloud
 - › Add spring-cloud-config-client dependency
- › Create a new application.properties file
- › Set the following properties
 - › spring.application.name
 - › spring.profiles.active
 - › spring.config.import



```
spring.application.name=user-microservice
spring.profiles.active=dev
spring.config.import=optional:configserver:http://localhost:8888
```

Configuration server URI

- › Run and test
 - › Run the microservice, change the port on the configuration file, stop and re-run the microservice

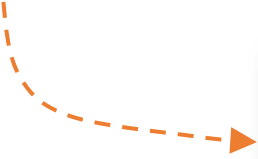
Update properties at runtime

Let's do a test

- › Change the user-microservice configuration
 - › Set the service port to, e.g., 9045
- › Restart user-service
 - › (We need to restart for updating its port)
- › Send a GET request to job microservice
 - › [GET] `http://localhost:9055/job/apply/applicant/2`
- › We should get an error
 - › Job service tries to invoke User service, but it is running on a different port
 - › We should update the configuration of Job service
 - › We need to restart the service for this?

Refresh and @RefreshScope

- › Using @RefreshScope annotation on a bean we can force it to refresh its configuration
 - › E.g., the @Value annotated properties
- › Add @RefreshScope to UserMicroserviceInvoker class
- › Add refresh to the actuator's exposed endpoints
 - › Into job-microservice-dev.properties
- › Invoke the actuator/refresh endpoint to update the configuration
 - › Send an empty POST request to localhost:9055/actuator/refresh
- › Try again to call without restarting



```
1  [
2    "microservice.user.find.uri"
3  ]
```

Running multiple instances

Running multiple instances

- › If we try to run each service more than once, we'll get an error

- › (If on the same host)
- › The port is already in use



```
*****  
APPLICATION FAILED TO START  
*****  
  
Description:  
  
Web server failed to start. Port 8888 was already in use.
```

- › Containerization solves this problem

- › Each service runs on its container, with port mapped

- › If we do not want this, we should provide different properties for each instance

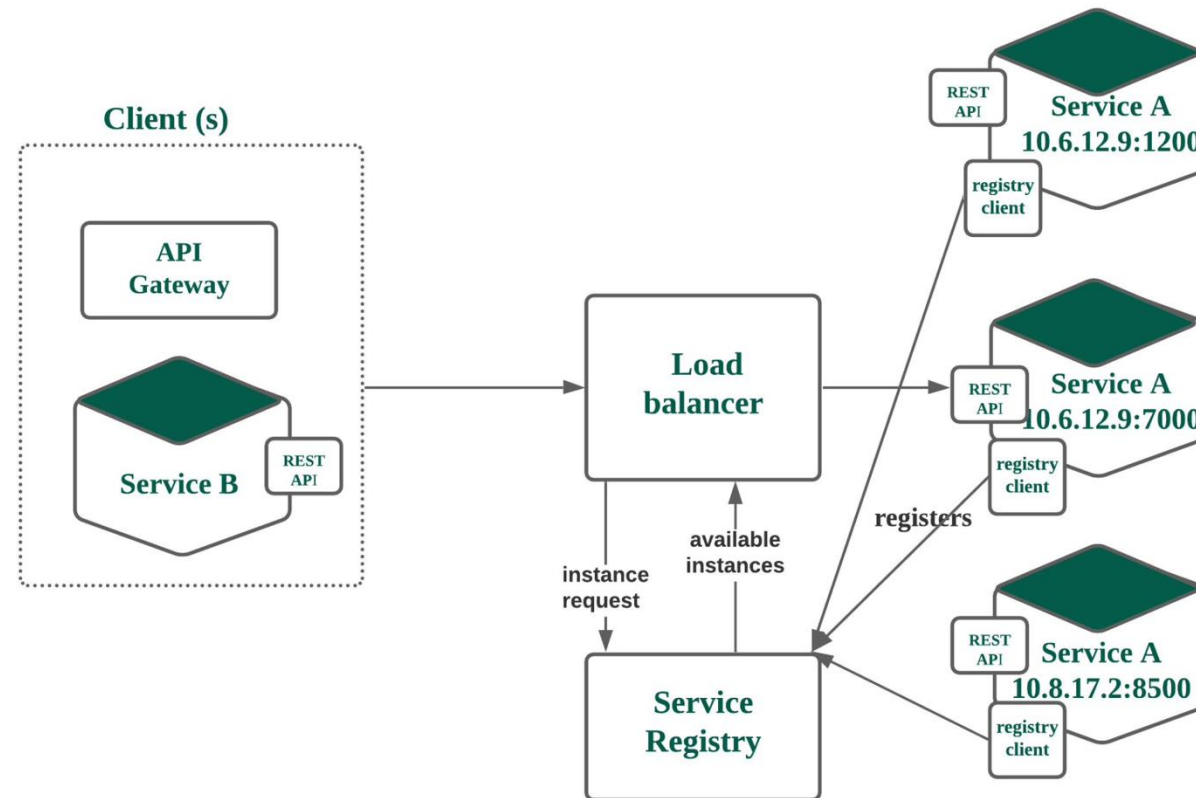
- › Does not sound good

- › We can override properties through command-line arguments

- › Command line arguments take precedence over file-based property sources
- › E.g., `mvn spring-boot:run -Dspring-boot.run.arguments="--server.port=9066"`

Load balancing

Load balancing microservices



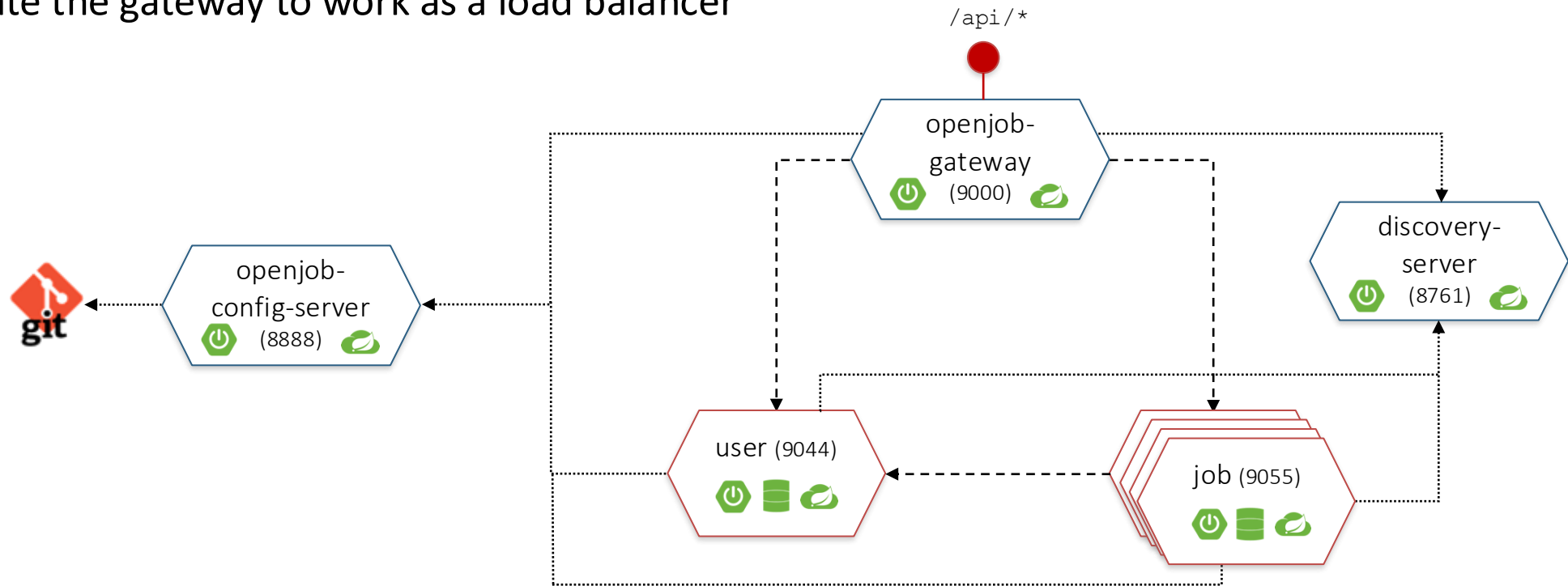
Spring Cloud Netflix

- › Spring Cloud Netflix provides tools to realize load balancing by integrating Netflix components
 - › Service discovery (Eureka)
 - › Netflix Eureka has served Netflix for years
 - › Previous version included Ribbon and Zuul for load balancing, Hystrix for fault tolerance
- › Eureka client
 - › Eureka client services register to the server and discover client instances
 - › Send heartbeats to the service
- › Eureka server
 - › Provides a home page with a UI and HTTP API endpoints for the normal Eureka functionality

Upgrade our Openjob system

Steps

- › Create a discovery server
- › Update services to work as eureka clients
- › Update the gateway to work as a load balancer



Eureka Server

New Spring Starter Project

Service URL:

Name:

☒ Use default location

Location:

Type: Packaging:

Java Version: Language:

Group:

Artifact:

Version:

Description:

Package:

Working sets

☐ Add project to working sets

Working sets:

New Spring Starter Project Dependencies

Spring Boot Version:

Frequently Used:

☐ Config Server ☐ Gateway ☐ H2 Database

☐ Spring Boot Actuator ☐ Spring Boot DevTools ☐ Spring Data JPA

☐ Spring Web

Available:

Selected:

X Eureka Server

▼ Spring Cloud Discovery

☐ Eureka Discovery Client

☒ Eureka Server

▼ VMware Tanzu Application Service

☐ Service Registry (TAS)

Eureka Server

- › Edit Application class
 - › Add @EnableEurekaServer annotation
- › We will run the server in the standalone mode
 - › I.e., only one Eureka server instance in our system
- › Edit application.yml file to add configuration
- › That's it
 - › Browse <http://localhost:8761/> to see the Eureka dashboard

```
@SpringBootApplication
@EnableEurekaServer
public class DiscoveryServerApplication {

    public static void main(String[] args) {
        SpringApplication.run(DiscoveryServerApplication.class, args);
    }

}
```

```
server:
  port: 8761

eureka:
  instance:
    hostname: localhost
  client:
    registerWithEureka: false
    fetchRegistry: false
    serviceUrl:
      defaultZone: http://${eureka.instance.hostname}:${server.port}/eureka/
```

Update services

Job service

- › Add spring-cloud-starter-netflix-eureka-client dependency to pom.xml

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-client</artifactId>
</dependency>
```

- › Add @EnableDiscoveryClient annotation to the Application class

```
@EnableJpaAuditing
@EnableDiscoveryClient
@SpringBootApplication
public class JobApplication {

    public static void main(String[] args) {
        SpringApplication.run(JobApplication.class, args);
    }

}
```

- › Run and browse again <http://localhost:8761/>

Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
JOB-MICROSERVICE	n/a (1)	(1)	UP (1) - localhost:job-microservice:9055

Play with Eureka service

- › Eureka server by default expects that clients send heartbeats every 30 seconds
 - › If clients have a different interval, a message is shown
- › To align clients to this interval, a configuration property is required
 - › `eureka.instance.lease-renewal-interval-in-seconds=30`
- › Clients unregister themselves at shutdown

```
Shutting down DiscoveryClient ...  
Unregistering ...  
DiscoveryClient_JOB-MICROSERVICE/localhost:job-microservice:9055 - deregister status: 200  
Completed shut down of DiscoveryClient
```

- › Let's test a network problem:
 - › Add the `eureka.client.should-unregister-on-shutdown=false` property to the client service
 - › Stop the service

Status
DOWN (1) - localhost:job-microservice:9055

Update the gateway

- › Same steps as before:
 - › Add spring-cloud-starter-netflix-eureka-client dependency to pom.xml
 - › Add @EnableDiscoveryClient annotation to the Application class
- › Add configuration properties
 - › Edit application.yml

```
eureka:  
  client:  
    registerWithEureka: true  
    healthcheck:  
      enabled: true  
    serviceUrl:  
      defaultZone: http://localhost:8761/eureka/  
  instance:  
    lease-renewal-interval-in-seconds: 30
```

Load balance instances

- › Update the gateway configuration
 - › Instruct the gateway to lookup services via the Eureka server

```
- id: job_route
  uri: http://localhost:9055
  predicates:
  - Path=/api/job/**
  filters:
  - RewritePath=/api(?<segment>/?), ${segment}
```

```
- id: job_route
  uri: lb://job-microservice
  predicates:
  - Path=/api/job/**
  filters:
  - RewritePath=/api(?<segment>/?), ${segment}
```

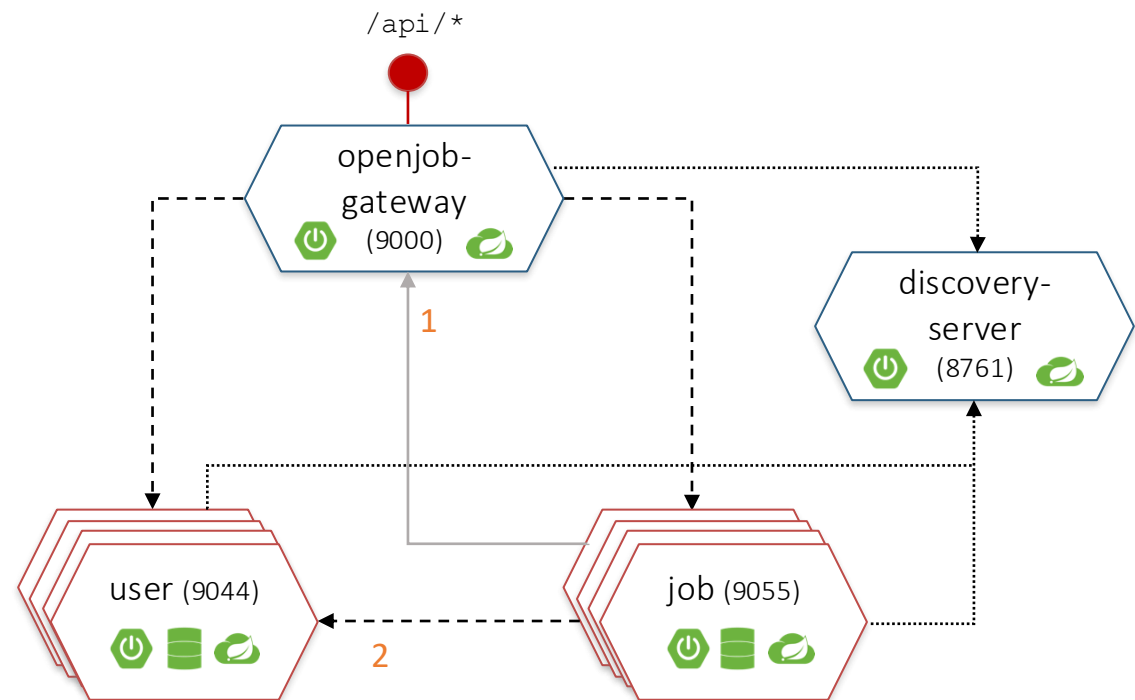
job-microservice must be defined through the
spring.application.name property in its service properties!!!

- › Test the load balancer
 - › Start (or POST to `http://localhost:9000/actuator/refresh`) the gateway
 - › Send a request to `http://localhost:9000/job` to test

Client-side load balancing

Client-side load balancer

- › When running multiple instances of the User microservice, load balancing is needed to route requests from Job microservice to User
- › First solution (1)
 - › Send requests through the API Gateway and its load balancer
 - › Realize a centralized approach
- › More decentralized solution (2)
 - › Use a client-side load balancer



Spring Cloud OpenFeign

- › Spring Cloud supports Feign, a declarative service client
 - › Feign creates a dynamic implementation of an interface decorated with JAX-RS annotations (e.g., `@RequestMapping`, etc.)
 - › Integrates discovery, load balancing, circuit breaking, etc. from Spring Cloud
- › Integrate OpenFeign
 - › Add `spring-cloud-starter-openfeign` pom dependency
 - › Add `@EnableFeignClients` annotation to Application class
- › Create a dynamic client
 - › Create an interface and annotate it with `@FeignClient` annotation
 - › Annotate methods with `@RequestMapping`

Update Job microservice

- › Job microservice has an invoker class to call User microservice
 - › Update and use a dynamic Feign client
- › Create a new package and a new interface
 - › Annotate the interface with @FeignClient
 - Name of the Feign client, url of the service to call

```
@FeignClient(name="userMicroserviceFeignClient", url="${microservice.user.find.uri}")  
public interface UserMicroserviceFeignClient {  
  
    @RequestMapping(method=RequestMethod.GET, value="{username}")  
    public JsonNode findUserByUsername(@PathVariable String username);  
}
```

- › Add a new method
 - › Annotate with @RequestMapping as in JAX-RS services
- › Update the controller class to use the Feign client
 - › Change the autowired service

Enable load balancing

- › In the current implementation we set the URL of the invoked service
 - › `@FeignClient(name="userMicroserviceFeignClient", url="${microservice.user.find.uri}")`

- › Feign supports Eureka service discovery
 - › Update the `@FeignClient` annotation to use the service discovery

```
@FeignClient(name="user-microservice")  
public interface UserMicroserviceFeignClient {
```

Application name of the user service

- › Update the `@RequestMapping` annotation

```
@RequestMapping(method=RequestMethod.GET, value="/user/username/{username}")  
public JsonNode findUserByUsername(@PathVariable String username);
```

Request path for the API call

Consistency issues with the databases

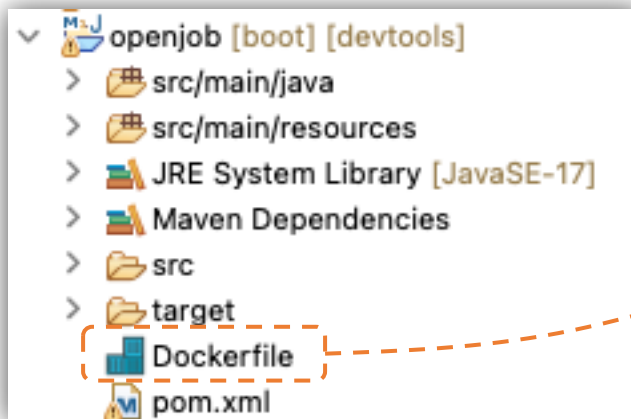
- › We are running the services with an H2 in-memory DB.
- › When running multiple instances, each instance will own its data
- › We need to switch to a standalone DB
- › Update User and Job microservices:
 - › Remove the H2 pom dependency, add mysql-connector-j
 - › Update properties file
 - › Remove the H2-related properties
 - › Update the data.sql to comply with the SQL syntax (e.g., use ` instead of ")

```
spring.datasource.url=jdbc:mysql://localhost:8889/jobdb?useSSL=false&serverTimezone=UTC&useLegacyDatetimeCode=false&createDatabaseIfNotExist=true
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.sql.init.mode=always
```

Deploying microservices on Docker

Create an image

- › Let's do it with the Openjob *monolith*
- › Create a Dockerfile on the root folder of the project



```
FROM maven:3.9.2-eclipse-temurin-17-alpine
WORKDIR /openjob
COPY . .
RUN mvn clean install
CMD mvn spring-boot:run
```

Base image for next instructions
It will be pulled from a public repo

Create the /openjob folder, move into it
and copy the content of the project inside

Compile the project

Specify the programs/commands to execute
while initializing a container from the image
(Run the openjob application)

- › Run the command `docker build -t openjob .` from terminal
- › Check the created image with `docker images` command (or from Docker Desktop)

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
openjob	latest	afdd82e565a6	26 seconds ago	534MB

Run the container

› Run the command `docker run --name openjob-app -p 9090:9092 -d openjob`

Name of the container to be created

Port mapping:

Host port 9090 mapped to container port 9092

Name of the image

Run in detached mode

› Run `docker container ls` to check the list of running containers

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
f703bf7be2e3	openjob	"/usr/local/bin/mvn-..."	5 seconds ago	Up 4 seconds	0.0.0.0:9090->9092/tcp	openjob-app

Dockerize the Mysql database

- › Create a network to let containers communicate
 - › `docker network create openjob-network`
- › Pull the mysql Docker image
 - › `docker pull mysql:latest`
- › Run the container
 - › `docker run -d --name openjob-mysql --network openjob-network -v openjob_mysql-data:/var/lib/mysql -e MYSQL_ROOT_PASSWORD=root mysql:latest`
- › Run a container with Phpmyadmin (not required, just to check)
 - › `docker pull phpmyadmin:latest`
 - › `docker run --name openjob-phpmyadmin -d --network openjob-network --link openjob-mysql:db -p 8081:80 phpmyadmin:latest`
- › Check at <http://localhost:8081>



Before dockerizing services...

Let's recall that

- › We are using a configuration server
 - › With configuration files stored in a git repository...
 - › Push files to a remote git repository (so it can be accessible from everywhere)
 - › I created my own, e.g., <https://github.com/gianlucafilippone/openjob-properties-repository>
- › When put on containers, service addresses should not be hard-coded
 - › We hard-coded the configuration server address
 - › ...the Eureka server
 - › ...the database address and parameters
- › Update properties files

```
eureka:  
  client:  
    registerWithEureka: true  
    healthcheck:  
      enabled: true  
    serviceUrl:  
      defaultZone: ${EUREKA_SERVER}  
      #defaultZone: http://localhost:8761/eureka/
```

```
#spring.config.import=optional:configserver:http://config-server:8888  
spring.config.import=optional:configserver:http://${CONFIG_SERVER_HOST}:${CONFIG_SERVER_PORT}  
spring.cloud.config.fail-fast=true
```

```
#spring.datasource.url=jdbc:mysql://localhost:8889/userdb?useSSL=false&serverTime  
spring.datasource.url=jdbc:mysql://${MYSQL_HOST}/${MYSQL_DB}?useSSL=false&allowPul  
spring.datasource.username=${MYSQL_USER}  
spring.datasource.password=${MYSQL_PASSWORD}  
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver  
#spring.sql.init.mode=always
```

Create Images

- › Create a Dockerfile for each maven project

```
FROM maven:3.9.2-eclipse-temurin-17-alpine

WORKDIR /config-server-app
COPY . .

RUN mvn clean install
CMD mvn spring-boot:run
```

```
FROM maven:3.9.2-eclipse-temurin-17-alpine

WORKDIR /discovery-server-app
COPY . .

RUN mvn clean install
CMD mvn spring-boot:run
```

```
FROM maven:3.9.2-eclipse-temurin-17-alpine

WORKDIR /gateway-app
COPY . .

RUN mvn clean install
CMD mvn spring-boot:run
```

```
FROM maven:3.9.2-eclipse-temurin-17-alpine

WORKDIR /job-microservice
COPY . .

RUN mvn clean install
CMD mvn spring-boot:run
```

```
FROM maven:3.9.2-eclipse-temurin-17-alpine

WORKDIR /user-microservice
COPY . .

RUN mvn clean install
CMD mvn spring-boot:run
```

- › Run the command `docker build -t <IMAGE-NAME> .` for each project
 - › Check the images by running `docker image list`

Run containers

- › Containers can be created and started with `docker run <IMAGE-NAME>`
 - › We must specify the environment variables to instantiate configuration parameters
- › Since running on the same network, containers can communicate through their name (instead of, e.g., localhost).
 - › Each container has its own IP address
- › Example
 - › `docker run --name user-microservice --network openjob-network -dp 9044:9044`
 - `-e CONFIG_SERVER_HOST=config-server`
 - `-e CONFIG_SERVER_PORT=8888`
 - `-e EUREKA_SERVER=http://discovery-server:8761/eureka/`
 - `-e MYSQL_HOST=openjob-mysql:3306`
 - `-e MYSQL_DB=user -e MYSQL_PORT=3306`
 - `-e MYSQL_USER=root`
 - `-e MYSQL_PASSWORD=root user-microservice`

Docker compose

- › When applications are made as a composition of many services, we can leverage docker compose instead of manually create images and run containers

```
services:
  openjob-mysql:
    image: mysql:latest
    volumes:
      - mysql-data:/var/lib/mysql
    environment:
      MYSQL_ROOT_PASSWORD: root

  openjob-phpmyadmin:
    image: phpmyadmin:latest
    ports:
      - "8081:80"
    links:
      - "openjob-mysql:db"
    depends_on:
      - openjob-mysql

  config-server:
    build: ./config-server
    ports:
      - "8888:8888"

  discovery-server:
    build: ./discovery-server
    ports:
      - "8761:8761"
    depends_on:
      - config-server
```

```
gateway:
  build: ./gateway
  ports:
    - "9000:9000"
  environment:
    CONFIG_SERVER_HOST: config-server
    CONFIG_SERVER_PORT: 8888
    EUREKA_SERVER: http://discovery-server:8761/eureka/
  depends_on:
    - config-server
    - discovery-server

user-microservice:
  build: ./user
  ports:
    - "9044:9044"
  environment:
    CONFIG_SERVER_HOST: config-server
    CONFIG_SERVER_PORT: 8888
    EUREKA_SERVER: http://discovery-server:8761/eureka/
    MYSQL_HOST: openjob-mysql:3306
    MYSQL_DB: user
    MYSQL_USER: root
    MYSQL_PASSWORD: root
  depends_on:
    - config-server
    - discovery-server
    - openjob-mysql
```

Running docker compose

- › Run the command `docker compose -p openjob up -d`
 - › Containers will be created and started
- › Make your application resilient!
 - › Dependencies ensure that containers are started in sequence
 - › Does not ensure that services are ready
 - › Use Spring Retry to get configuration and avoid failures
 - › Add `spring-retry` and `spring-boot-starter-aop` dependencies
- › Scale services dynamically
 - › Use `docker-compose up --scale <SERVICE-NAME>=<NUMBER> -d`
 - › Update the `docker-compose.yml` file to make the port selection dynamic
 - Use "expose" to randomly map host ports to the exposed container port
 - Select a range of host ports to be mapped to the exposed container port (e.g., "9044-9049:9044")